

Smart Lead Assignment & Follow-Up Engine

Project documentation

Candidate: Gowtham Jegatheeswaran

Organisation: Neirah Tech Solution (internship task)

Stack: Node.js 22, NestJS 11, TypeScript, PostgreSQL 16, TypeORM, Docker, JWT, Jest

Repository: <https://github.com/GowthamJegatheeswaran/neirah-crm-engine>

Contents

1. What this project does
2. Technology choices
3. Architecture
4. Database design (with ER diagram)
5. Authentication and roles
6. Lead and employee management
7. Assignment engine
8. Follow-ups, SLA and escalation
9. History and audit
10. API reference
11. Testing
12. Setup and running
13. Demo helpers
14. Known limits and next steps

1. What this project does

This is the backend of a small CRM. Leads arrive from the website, referrals, campaigns, social media or manual entry. The system decides which sales employee should get each lead, tracks follow-ups, and escalates leads that nobody has acted on in time. Every decision is stored, so it is always possible to see who got a lead, why, and what happened afterwards.

The brief asked for four things, and this is where each one lives:

Requirement	How it is done
Lead management	Create, edit, search, filter and paginate leads. Every change adds a row to an activity timeline.
Smart assignment	Rules stored in the database. The engine filters by specialization, territory, availability and workload, then ranks the candidates.
Follow-ups	Calls, meetings and emails with a due date. A follow-up becomes overdue automatically and moves with the lead if it is reassigned.
Escalation / SLA	SLA policies stored in the database. If nobody responds in time, the lead is flagged or moved to another employee.

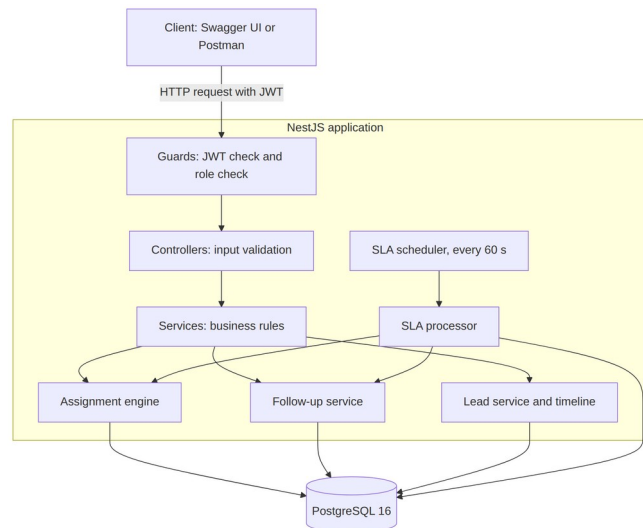
Nothing about assignment or escalation is hard-coded. Rules and SLA policies are rows in tables, so an admin can change behaviour through the API without a new deployment.

2. Technology choices

Choice	Why
NestJS + TypeScript	Modules, dependency injection and guards keep a larger API organised. TypeScript catches many mistakes before running.
PostgreSQL	I needed transactions, row locks, advisory locks and unique constraints. The SLA idempotency and the assignment locking depend on them.
TypeORM with migrations	The schema changes only through migration files (synchronize is switched off), so any clone can rebuild the same database.
JWT + role guards	Stateless login. A global guard protects every route unless it is marked public.
Swagger / OpenAPI	Generated from the code, so the documentation cannot drift away from the API.
Docker Compose	One command starts the database and the API. The API container runs the migrations by itself on start.
Jest + Supertest	Unit tests for the pure logic and end-to-end tests that call the real API against a real database.

3. Architecture

A request goes through the same path every time. First the guards check the token and the role. Then the controller validates the input. Then a service applies the business rules and talks to the database. Errors from any step go through one exception filter, so every error has the same JSON shape.



The SLA scheduler runs inside the same application. Every 60 seconds it calls the same SLA processor that the endpoint POST /sla/run calls, so the manual button and the background job use identical code.

Layers, from outside to inside: controller (HTTP only), service (business logic), repository or entity (data access). The assignment matching and the SLA breach check are plain functions with no database access, which is why they are easy to unit test.

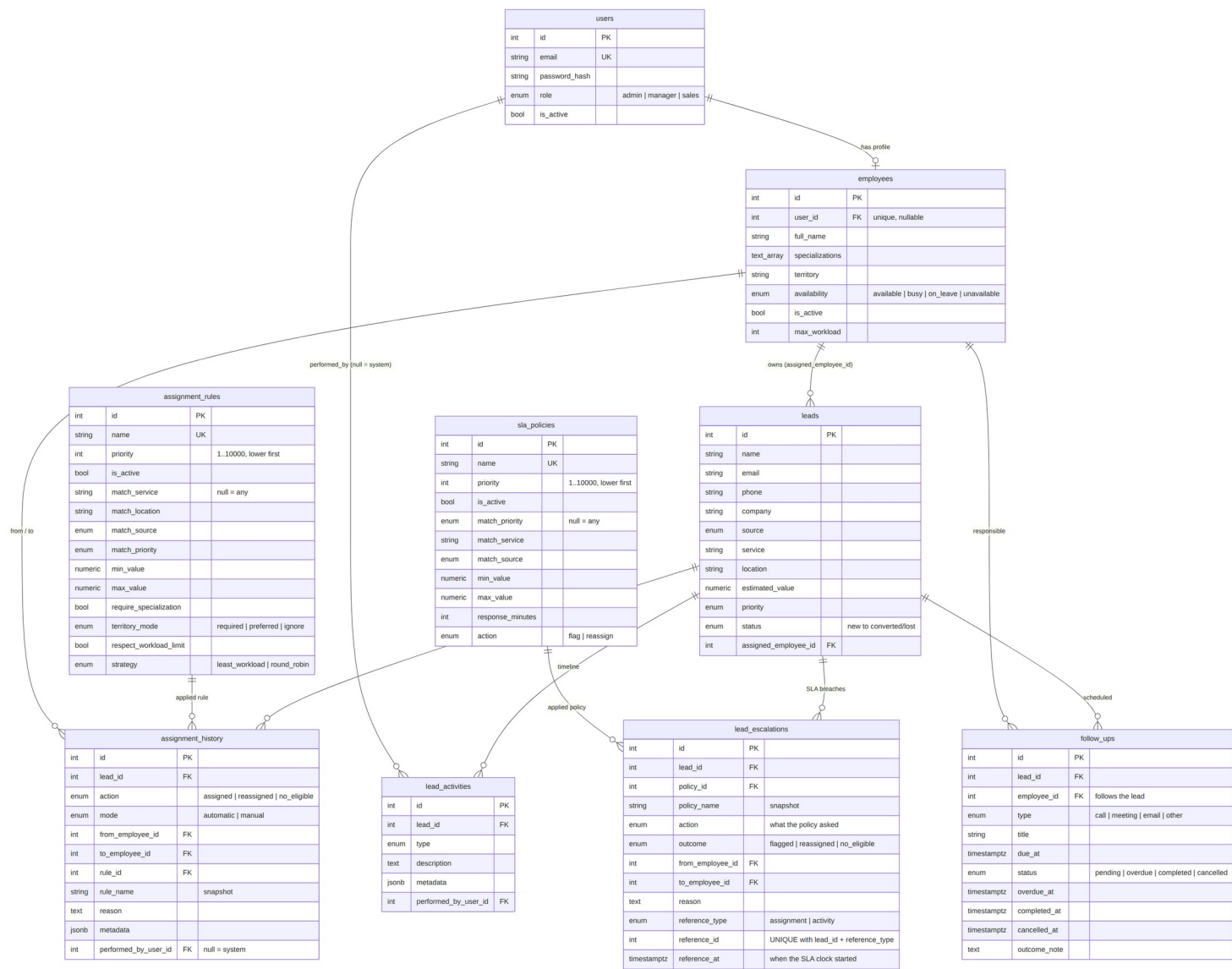
4. Database design

There are nine tables. The full ER diagram is on the next page.

Table	Purpose
users	Login accounts with a role: admin, manager or sales. Passwords are stored as bcrypt hashes.
employees	Sales profile: specializations, territory, availability, active flag and maximum workload. Linked to a user.
leads	The leads, with source, service, location, estimated value, priority, status and the assigned employee.
lead_activities	The timeline. Append-only: rows are only ever inserted.
assignment_rules	Configurable assignment rules.
assignment_history	Every assignment decision, automatic or manual, with the reason. Append-only.
follow_ups	Follow-up tasks with status pending, overdue, completed or cancelled.
sla_policies	Configurable SLA policies.
lead_escalations	One row per SLA breach that was handled. A unique key on this table stops duplicates.

Decisions worth knowing: workload is not stored on the employee. It is counted from the open leads, so it can never go out of date. History tables keep a copy of the rule or policy name, so old records still make sense after a rule is edited. Leads are never deleted through the API; a lost lead gets the status "lost".

ER diagram



5. Authentication and roles

A user logs in with email and password and receives a JWT. The token is sent as a Bearer header. On every request the user is loaded again from the database, so a deactivated user is blocked immediately even if the token has not expired.

Role	What they can do
admin	Everything, including creating users, assignment rules and SLA policies.
manager	Create and manage leads and employees, assign and reassign leads, run the SLA check, see all dashboards. Cannot create users, rules or policies.
sales	See and work only on their own leads and follow-ups, change their own availability, see their own dashboard.

Unauthenticated requests get 401. A logged-in user without permission gets 403. If a sales user asks for a lead that belongs to someone else, the answer is 404 instead of 403, so lead ids cannot be guessed.

Other protection: password rules, a generic login error (no hint whether the email exists), request validation that rejects unknown fields, Helmet security headers, and no secrets in the repository.

6. Lead and employee management

Leads

A lead has a name, optional contact details, a source, a service, a location, an estimated value and a priority. The list endpoint supports search, filters (status, priority, source, service, location, owner, value range, dates), sorting and pagination (maximum 100 per page). Sort fields are a fixed whitelist and search text is escaped.

Status flow: new → assigned → contacted → qualified → follow_up → converted or lost. The system sets "new" and "assigned"; a person cannot set them. "converted" and "lost" are final. An invalid jump returns 409.

Every change (creation, edit, status change, note, assignment, follow-up event, escalation) writes a row to the timeline in the same database transaction as the change itself. The timeline therefore cannot disagree with the data.

Employees

An employee has specializations (a list), a territory, an availability (available, busy, on_leave, unavailable), an active flag and a maximum workload. Managers can create and edit profiles. A sales user can change only their own availability.

7. Assignment engine

When a lead is created (or when a manager runs the engine on a "new" lead) the engine follows these steps:

1. Go through the assignment rules in order of priority number (lowest first). The first active rule whose conditions match the lead is used. A condition that is left empty means "any". Conditions are service, location, source, priority and value range.
2. Apply the hard filters, always: the employee is active, availability is "available", the linked user is active, and the lead's service is among the employee's specializations (if the rule asks for it).

3. Apply the rule policy: skip employees who are at their maximum workload, and treat territory as required, preferred or ignored.
4. Rank what is left. Strategy "least_workload": fewest open leads first, then the one who was assigned least recently, then the lowest id. Strategy "round_robin": least recently assigned, then lowest id. There is no randomness, so the result can be predicted and tested.
5. Save the result with a readable reason, for example: Rule "Default: specialization + preferred territory": Arun Kumar chosen by lowest workload (0/10 open leads) among 3 eligible.
6. If nobody qualifies, the lead stays "new" and a no_eligible record is written. Nothing crashes.

Preview: GET /leads/{id}/assignment-preview runs the same logic without saving anything. It shows the candidates, who was excluded and why, and which tie-break was used.

Manual reassignment: POST /leads/{id}/reassign with an employee id and a reason. The old history stays; a new row is added. If the manager picks someone who is on leave or full, it is allowed but the warning is recorded.

Concurrency: a PostgreSQL advisory lock makes assignment decisions run one at a time, and the lead row is locked while it is changed. Two simultaneous requests therefore cannot overfill one employee or assign the same lead twice.

Seed rules: the demo data has three rules: a high-value rule that requires the same territory, a disabled round-robin rule, and a default rule (specialization plus preferred territory).

8. Follow-ups, SLA and escalation

Follow-ups

A follow-up has a title, a type (call, meeting, email, other), a due date and a status. The due date must be in the future when it is created. States: pending, overdue, completed, cancelled. When a lead is reassigned, its open follow-ups move to the new owner. When a lead becomes converted or lost, its open follow-ups are cancelled automatically.

SLA policies

A policy says: leads that match these conditions (priority, service, source, value range) must get a response within a number of minutes. If not, take an action: "flag" (only record it) or "reassign" (move the lead to someone else). Like rules, policies are checked by priority number and the first active match wins.

The seed adds three: high-priority leads must be answered in 30 minutes, then reassigned; high-value leads in 2 hours, flag only; everything else in one day, flag only.

How a breach is found

The SLA clock starts at the later of two moments: the last assignment, or the last response on the lead. A response means a status change, a note, or a follow-up being created or completed. A lead that has a pending follow-up which is not yet due counts as covered. Only open leads that have an owner are checked.

What happens on a breach

1. Late follow-ups are marked overdue first, each with one activity entry.
2. For every breached lead, one transaction writes "sla_breached" and "escalated" activities and a row in lead_escalations.

3. With the "reassign" action, the normal assignment engine picks another eligible employee. It never picks the current owner. The history shows action "reassigned", mode "automatic", performed by the system.
4. If nobody else is eligible, the owner is kept and the escalation is saved with outcome "no_eligible".

Running it twice does not repeat anything

A breach is identified by the event that started the clock (an assignment id or an activity id). The table `lead_escalations` has a unique key on the lead, the reference type and the reference id, so the same breach can be stored only once. A new assignment or a new response creates a new reference and opens a fresh window. Each lead is also re-checked under a lock before any action, and an advisory lock allows only one processor run at a time, even with several application instances.

Triggers: the scheduler (every 60 seconds, configurable, can be turned off) and `POST /sla/run` (managers and admins). The endpoint accepts `dryRun=true` to report what would happen without changing anything.

9. History and audit

- **Activity timeline** (`lead_activities`): everything that happens to a lead, in order, with who did it (empty means the system).
- **Assignment history** (`assignment_history`): every decision, automatic or manual, including failed ones, with rule name, reason, candidates and exclusions.
- **Escalations** (`lead_escalations`): every SLA breach that was handled, with the policy, the outcome and the employees involved.

All three are append-only. Nothing is overwritten, so a lead's whole story can be reconstructed.

Dashboard

Managers and admins get an overview (leads by status, source and priority, unassigned count, conversion and win rate, follow-up counts, escalations, assignment counts) and a per-employee table (open leads, utilisation, converted and lost, pending and overdue follow-ups, SLA breaches). Sales users get a personal view with their numbers and their next follow-ups.

10. API reference

The complete, always up-to-date reference is Swagger at `/api/docs`. A copy is saved as `docs/openapi.json`, and `docs/postman_collection.json` can be imported into Postman (run `POST /auth/login`, then put the token in the "token" variable). Every operation lists its possible error responses (400, 401, 403, 404). The main endpoints are:

Endpoint	Access	Purpose
<code>POST /auth/login</code>	Public	Log in, get a token
<code>GET /auth/me</code>	Any user	Current user
<code>POST /users, GET /users</code>	Admin / Admin, Manager	Create and list users
<code>POST /employees, GET /employees</code>	Admin, Manager	Create and list employees
<code>GET, PATCH /employees/{id}</code>	Admin, Manager	Read and edit an employee

Endpoint	Access	Purpose
GET /employees/me, PATCH /employees/me/availability	Sales	Own profile and availability
POST /leads	Admin, Manager	Create a lead (auto-assigns)
GET /leads	All (sales: own)	Search, filter, sort, paginate
GET, PATCH /leads/{id}	All (sales: own)	Read and edit a lead, change status
POST /leads/{id}/notes	All (sales: own)	Add a note
GET /leads/{id}/activities	All (sales: own)	Timeline
POST /leads/{id}/assign	Admin, Manager	Run the engine for a new lead
GET /leads/{id}/assignment-preview	Admin, Manager	Dry run, nothing saved
POST /leads/{id}/reassign	Admin, Manager	Manual reassignment
GET /leads/{id}/assignment-history	All (sales: own)	Assignment history
POST, GET /assignment-rules	Admin / Admin, Manager	Create and list rules
GET, PATCH /assignment-rules/{id}	Admin, Manager	Read, edit, activate or deactivate
POST, GET /leads/{id}/follow-ups	All (sales: own)	Create and list follow-ups of a lead
GET /follow-ups	All (sales: own)	Filter and paginate follow-ups
GET, PATCH /follow-ups/{id}	All (scoped)	Read, edit, reschedule
POST /follow-ups/{id}/complete, /cancel	All (scoped)	Close a follow-up
POST, GET /sla-policies	Admin / Admin, Manager	Create and list SLA policies
GET, PATCH /sla-policies/{id}	Admin, Manager	Read and edit a policy
POST /sla/run	Admin, Manager	Run the SLA check (supports dryRun)
GET /escalations, GET /leads/{id}/escalations	Admin, Manager / All (sales: own)	Escalation history
GET /dashboard/overview, /employees	Admin, Manager	Statistics
GET /dashboard/me	Sales	Personal statistics
GET /health	Public	API and database status

Errors always look like this: statusCode, error, message (a text or a list of validation messages), path and timestamp. Stack traces and SQL are never returned.

11. Testing

62 unit tests cover the pure logic: rule matching, candidate ranking and tie-breaks, SLA policy matching and breach detection, status rules, and the auth service.

137 end-to-end tests call the real API against a real PostgreSQL database. Each test builds its own data with a unique marker and cleans up afterwards, so tests do not depend on the seed data or on each other. To test time-based behaviour, the tests move rows into the past with SQL instead of waiting.

Brief requirement	Where it is tested
Full flow: create, assign, follow-up, escalate	day2, day3 and day4 end-to-end tests
Several eligible employees, different workloads	day3 end-to-end and unit tests
No match for specialization or territory	day3 end-to-end tests
Inactive or unavailable employees excluded	day3 end-to-end tests
Equal workload tie-break	day3 end-to-end and unit tests
Manual reassignment, history preserved	day3 end-to-end tests
Overdue follow-up and no-action SLA	day4 end-to-end tests
Repeated scheduler runs, no duplicates	day4 end-to-end tests
Invalid payloads, unauthorised access, missing resources	day2 to day5 end-to-end tests
Search, filter, pagination, dashboard	day2, day4 and day5 end-to-end tests
Token abuse, hostile input, role matrix	day5 end-to-end tests (28 tests)

Run them with `npm test` and `npm run test:e2e`. A clean clone with a fresh database passes both.

12. Setup and running

Requirements: Node.js 20 or newer, npm, Docker.

Option A: everything in Docker

```
git clone https://github.com/GowthamJegatheeswaran/neirah-crm-engine.git
cd neirah-crm-engine
cp .env.example .env # then set your own DB password and JWT secret
docker compose up -d --build
npm install && npm run seed # optional demo data
```

The API container applies the migrations before it starts. It listens on port 3000.

Option B: database in Docker, API on the machine

```
npm install
cp .env.example .env
docker compose up -d db
npm run migration:run
npm run seed
npm run start:dev
```

Open <http://localhost:3000/api/docs> for Swagger and <http://localhost:3000/health> for the status check.

Configuration

Variable	Meaning
PORT	API port, default 3000

Variable	Meaning
DB_HOST, DB_PORT, DB_USERNAME, DB_PASSWORD, DB_NAME	PostgreSQL connection
JWT_SECRET, JWT_EXPIRES_IN	Token signing secret and lifetime
SCHEDULER_ENABLED	true by default. Turns the background SLA job on or off
SLA_CHECK_INTERVAL_SECONDS	How often the SLA job runs, default 60
SEED_DEFAULT_PASSWORD	Password for the demo users, used only by the seed

The .env file is never committed; .env.example shows the names only. The seed script refuses to run when NODE_ENV is production, and the demo data is kept in its own file (src/database/seeds/demo-data.ts), separate from the application code.

Demo accounts

Created by `npm run seed`. The password is the value of `SEED_DEFAULT_PASSWORD`. `admin@neirah.test` (admin), `manager@neirah.test` (manager), and sales users `arun@`, `priya@`, `kumar@`, `divya@`, `nimal@` and `sara@neirah.test`. In the seed data one employee is on leave, one is inactive, and one lead (service "Government", location "Galle") has nobody who can take it, so the "no eligible employee" case can be shown.

13. Demo helpers

- `npm run demo`: runs a narrated check of the whole system against the running API (login and roles, workload assignment, tie-break, no match, inactive employee, manual reassignment, overdue follow-up, SLA escalation, repeated runs, search and dashboard). It uses its own data and removes it afterwards.
- `npm run time-travel -- lead <id> <minutes>` and `npm run time-travel -- followup <id> <minutes>`: make a lead or follow-up older so overdue and SLA cases can be shown without waiting. It changes timestamps in the database only and refuses to run in production.
- `npm run docs:export`: regenerates docs/openapi.json from the code.

The step-by-step script for the video, with the exact requests, is in docs/DEMO_SCRIPT.md.

14. Known limits and next steps

- The SLA processor loads all candidate leads in one query. That is fine for this size; for a very large dataset it should work in batches.
- The scheduler is a simple timer inside the application. It is safe with several instances because of the advisory lock, but a job queue would suit a bigger deployment.
- There is no refresh-token flow and no password reset. Tokens simply expire (default 1 hour).
- Notifications (email, SMS, WhatsApp) are outside the scope of the brief. Escalations are recorded and visible through the API, and a notification service could subscribe to them later.
- Leads cannot be deleted through the API on purpose, to keep the history intact. A data-retention policy would be needed in a real system.
- The docker-compose file is meant for development and demonstration. A production deployment would need secrets management, HTTPS and backups.